PRECISION AUTOMATION FRAMEWORK

API Module — Technical Documentation

Prepared By	**P. Mohan Sri Sai Surya**
Role	API Module Lead
Module	`api-module/`
Application Under Test	AutomationExercise.com
Tech Stack	Java 17, RestAssured, Cucumber, TestNG, ExtentReports

1. Module Overview

The api-module is a dedicated REST API test automation module built as part of the Precision Automation Framework — a Maven multi-module project. This module is responsible for validating all backend REST API endpoints exposed by the AutomationExercise e-commerce application.

The module follows a clean layered architecture separating concerns into API client configuration, request building, response validation, step definitions, hooks, and Cucumber runners. All test scenarios are written in BDD Gherkin syntax and executed via TestNG.

1.1 Objectives

- Validate all critical REST API endpoints for products and user account management
- Ensure correct HTTP status codes and response body messages are returned
- Cover both positive and negative test scenarios
- Provide detailed test execution reports via ExtentReports (Spark)
- Maintain test data and configuration independently via config.properties

1.2 Application Under Test — API Base URL

Base URL: https://automationexercise.com

Method	Endpoint	Description	Key Validation
GET	/api/productsList	Fetch all products	Status 200, non-empty list
POST	/api/searchProduct	Search product by name	Status 200, results not empty
POST	/api/createAccount	Create user account	responseCode 201, 'User created!'
PUT	/api/updateAccount	Update user account	responseCode 200, 'User updated!'
DELETE	/api/deleteAccount	Delete user account	responseCode 200, 'Account deleted!'
POST	/api/searchProduct	Search without param (negative)	responseCode 400, error message

2. Module Architecture & Package Structure

The api-module follows a strict layered package structure under com.precision.api. Each layer has a single responsibility, ensuring clean separation of concerns and maximum reusability.

2.1 Package Structure

```
api-module/
  src/
    main/java/com/precision/api/
      client/          ApiClient.java
      builders/        ProductRequestBuilder.java
                       UserRequestBuilder.java
      validators/      ProductValidator.java
                       UserValidator.java
    test/java/com/precision/api/
      steps/           ProductSteps.java
                       UserSteps.java
      hooks/           ApiHooks.java
      runner/          ApiTestRunner.java
    test/resources/features/
                       get_products.feature
                       search_product.feature
                       create_user.feature
                       update_user.feature
                       delete_user.feature
                       negative_validation.feature
```

2.2 Layer Responsibilities

Layer	Class	Responsibility
Client	ApiClient	Builds and caches the singleton RestAssured RequestSpecification with base URL, content type, and accept headers
Builders	ProductRequestBuilder	Constructs and fires HTTP requests to product endpoints (GET all, POST search, POST search without param)
Builders	UserRequestBuilder	Constructs and fires HTTP requests for user CRUD — createAccount, updateAccount, deleteAccount with form params

Validators	ProductValidator	Asserts response code, validates product list is non-empty, validates search results, validates 400 + error for negative test
Validators	UserValidator	Asserts responseCode and message for create (201), update (200), and delete (200) user operations
Steps	ProductSteps	Cucumber step definitions for product-related scenarios, wiring feature file steps to request builders and validators
Steps	UserSteps	Cucumber step definitions for user CRUD scenarios with static shared state for email/password across dependent steps
Hooks	ApiHooks	@Before creates ExtentTest per scenario; @After marks pass/fail and flushes the report — no browser involved
Runner	ApiTestRunner	TestNG + Cucumber runner pointing to all 6 feature files with glue configuration and HTML report output

3. Class-by-Class Technical Documentation

3.1 ApiClient.java

Package: com.precision.api.client

ApiClient provides a lazily initialized, singleton RequestSpecification used by all request builders. This prevents redundant configuration setup on every request and ensures all HTTP calls share the same base configuration.

Key Design Decisions

- Singleton pattern — RequestSpecification is created once and reused across all tests
- Base URI is loaded from config.properties via ConfigReader using key base.api.url
- Content-Type is set to URLENC (application/x-www-form-urlencoded) because all API endpoints accept form parameters
- Accept header is set to JSON so responses are parsed as JSON by RestAssured
- Private constructor prevents external instantiation

Method

getBaseSpec()	Returns the singleton RequestSpecification. Creates it on first call using RequestSpecBuilder with base URI, content type, and accept headers.

3.2 ProductRequestBuilder.java

Package: com.precision.api.builders

ProductRequestBuilder is a stateless utility class that constructs and fires HTTP requests to the product-related API endpoints. All methods return the raw RestAssured Response object for downstream validation.

Methods

Method	HTTP Method	Description
getAllProducts()	GET	Sends GET /api/productsList with no params. Returns complete product list response.
searchProduct(name)	POST	Sends POST /api/searchProduct with form param search_product=name. Used for positive search test.

searchProductWithNoParam()	POST	Sends POST /api/searchProduct with NO parameters. Used for negative test to validate 400 error response.

3.3 UserRequestBuilder.java

Package: com.precision.api.builders

UserRequestBuilder handles all user account operations. It reads static user profile data (name, address, DOB, etc.) from config.properties and accepts dynamic email and password parameters to support unique user creation per test run.

Methods

Method	HTTP	Form Parameters
createUser(email, password)	POST	name, email, password, title, birth_date/month/year, firstname, lastname, company, address1/2, country, zipcode, state, city, mobile_number — all from config
updateUser(email, password)	PUT	name (updated.name), email, password, firstname (updated.firstname), lastname (updated.lastname) — updated values from config
deleteUser(email, password)	DELETE	email, password — uses credentials established during createUser call

Unique Email Strategy

A UUID-based email (mohansurya+<UUID>@test.com) is generated in UserSteps before each create call, ensuring no test run conflicts with previously created accounts. The email and password are stored as static class-level variables in UserSteps and shared across the dependent create, update, and delete steps.

3.4 ProductValidator.java

Package: com.precision.api.validators

ProductValidator contains static assertion methods for product API responses. All validations use Java assert statements and log both expected and actual values before asserting.

Method	Validation Logic
validateStatusCode(response)	Extracts responseCode from JSON body. Asserts it equals 200.
validateProductsList(response)	Extracts products JSON array. Asserts it is non-null and has at least one item.
validateSearchResults(response)	Extracts products from search response. Asserts non-null and non-empty list.
validateMissingParameter(response)	Asserts responseCode is 400. Asserts message equals exact expected string: 'Bad request, search_product parameter is missing in POST request.'

3.5 UserValidator.java

Package: com.precision.api.validators

UserValidator validates user account API responses. It parses the raw response string using JsonPath for reliable extraction independent of HTTP status codes.

Method	Expected Code	Expected Message
validateUserCreated(response)	201	User created!
validateUserUpdated(response)	200	User updated!
validateUserDeleted(response)	200	Account deleted!

3.6 ProductSteps.java

Package: com.precision.api.steps

ProductSteps wires Cucumber Gherkin steps to the ProductRequestBuilder and ProductValidator. A single Response field stores the result of each request for subsequent validation steps.

Step Definition	Action
the API is available	Logs step entry — no action, confirms API precondition
user sends GET request to products list	Calls ProductRequestBuilder.getAllProducts()
response status code should be 200	Calls ProductValidator.validateStatusCode(response)
products list should not be empty	Calls ProductValidator.validateProductsList(response)
user searches for product {string}	Calls ProductRequestBuilder.searchProduct(productName)
search results should not be empty	Calls ProductValidator.validateSearchResults(response)
user sends POST request to search without any parameter	Calls ProductRequestBuilder.searchProductWithNoParam()
response should indicate missing parameter	Calls ProductValidator.validateMissingParameter(response)

3.7 UserSteps.java

Package: com.precision.api.steps

UserSteps manages the stateful user CRUD flow. Since create, update, and delete steps share the same user credentials, static class-level variables are used to pass data between steps within the same test run.

State Management Design

- sharedEmail — static String, generated fresh per test run using UUID
- sharedPassword — static String, loaded from config.properties (key: valid.password)
- userCreated — static boolean flag, prevents update/delete from running if create was never called

Step	Action
user sends POST request to create account with valid details	Generates UUID email, reads password from config, calls UserRequestBuilder.createUser(), sets userCreated=true
create response status code should be 200	Asserts HTTP status code equals 200 directly on response
response message should confirm user created	Calls UserValidator.validateUserCreated(response)

user sends PUT request to update account with new details	Guards with userCreated flag, calls UserRequestBuilder.updateUser() with shared credentials
update response status code should be 200	Asserts responseCode from JSON body equals 200
response message should confirm user updated	Calls UserValidator.validateUserUpdated(response)
user sends DELETE request with valid credentials	Guards with userCreated flag, calls UserRequestBuilder.deleteUser(), sets userCreated=false
delete response status code should be 200	Asserts responseCode from JSON body equals 200
response message should confirm account deleted	Calls UserValidator.validateUserDeleted(response)

3.8 ApiHooks.java

Package: com.precision.api.hooks

ApiHooks manages the ExtentReport lifecycle for each API test scenario. Unlike the UI module, no browser is involved — there is no WebDriver initialization or screenshot capture.

Hook	Action
@Before	Calls ExtentTestManager.createTest(scenario.getName()) to register a new test node in the ExtentReport
@After	If scenario failed: logs error, marks ExtentTest as failed. If passed: marks as passed. Always calls removeTest() and flushReports()

3.9 ApiTestRunner.java

Package: com.precision.api.runner

ApiTestRunner is the single TestNG entry point for all API tests. It extends AbstractTestNGCucumberTests and uses @CucumberOptions to configure feature paths, glue packages, and report plugins.

- Features: 6 files — get_products, search_product, create_user, update_user, delete_user, negative_validation
- Glue packages: com.precision.api.steps and com.precision.api.hooks
- Plugins: html:target/cucumber-reports/api-report.html
- Parallel execution: disabled (parallel = false) to preserve state-dependent user CRUD ordering
- Monochrome: true for clean console output

4. Test Scenarios & Feature Files

The api-module contains 6 Cucumber feature files covering 6 distinct test scenarios. Each feature file maps directly to one API endpoint or one validation concern.

#	Feature File	Scenario	Endpoint	Type	Expected Result
1	get_products.feature	Get all products list	GET /api/productsList	Positive	200, non-empty list
2	search_product.feature	Search product by name 'tshirt'	POST /api/searchProduct	Positive	200, results returned
3	create_user.feature	Create new user account	POST /api/createAccount	Positive	201, User created!
4	delete_user.feature	Delete existing user account	DELETE /api/deleteAccount	Positive	200, Account deleted!
5	update_user.feature	Update existing user details	PUT /api/updateAccount	Positive	200, User updated!
6	negative_validation.feature	Search product without parameter	POST /api/searchProduct	Negative	400, missing param msg

4.1 Dependent Test Ordering

The Create → Update → Delete user flow is intentionally order-dependent. The UserSteps class uses a static userCreated flag and shared static email/password fields to pass context across scenarios without Cucumber dependency injection. If Update or Delete is triggered before Create, the userCreated guard throws an IllegalStateException with a descriptive message.

5. Execution Instructions

5.1 Prerequisites

- Java 17 or above installed
- Maven 3.8+ installed
- config.properties configured with base.api.url, valid.password and all user.* fields

5.2 Run Command

```
mvn test -pl api-module
```

5.3 Run Specific Feature

```
mvn test -pl api-module -Dcucumber.filter.tags="@products"
```

5.4 Report Location

```
target/extent-reports/AutomationReport.html   (ExtentReports - Dark Theme)
target/cucumber-reports/api-report.html       (Cucumber HTML Report)
```

5.5 config.properties Keys Used by api-module

Key	Purpose
base.api.url	Root URL for all API requests (e.g. https://automationexercise.com)
valid.password	Password used when creating and authenticating test user accounts
user.name	Full name field for createAccount form parameter
user.title	Title (Mr/Mrs) for createAccount form parameter
user.firstname	First name for createAccount and updateAccount
user.lastname	Last name for createAccount and updateAccount
user.birth_date/month/year	Date of birth fields for createAccount
user.address1/2, city, state, country, zipcode	Address fields for createAccount
user.mobile_number	Mobile number for createAccount

updated.name, updated.firstname, updated.lastname	Updated values used in PUT updateAccount request

6. Challenges Faced & Solutions

Challenge	Solution
Shared state between Create, Update, and Delete steps — each step is independent in Cucumber but needs email/password from the Create step	Used static class-level fields (sharedEmail, sharedPassword, userCreated) in UserSteps. A boolean guard flag throws IllegalStateException if dependent steps run out of order.
HTTP status code vs responseCode in JSON body — the API returns HTTP 200 for createAccount but responseCode 201 inside the JSON body	Handled in validators by distinguishing between response.getStatusCode() (HTTP level) and response.jsonPath().getInt('responseCode') (JSON body level). Both are validated separately.
Unique user per test run — reusing same email across runs causes 'User already exists' failures	Generated UUID-based unique email (mohansurya+<UUID>@test.com) before each createUser call, guaranteeing uniqueness per execution.
Test ordering for dependent scenarios — Cucumber does not guarantee feature file execution order	Listed feature files in ApiTestRunner in the required sequence: create_user.feature before update_user.feature and delete_user.feature. Parallel execution is disabled.
Negative test for missing parameter — needed to confirm exact error message, not just status code	validateMissingParameter() in ProductValidator asserts both the responseCode (400) and the full exact message string using assertEquals for precise validation.

7. Conclusion & Future Enhancements

7.1 Summary

The api-module successfully automates all critical REST API endpoints of the AutomationExercise platform. The layered architecture (client → builders → validators → steps → hooks → runner) ensures clean separation of concerns, high reusability, and ease of maintenance.

- 6 test scenarios covering GET, POST, PUT, DELETE, and negative validation
- Singleton ApiClient ensures consistent request configuration across all tests
- UUID-based unique email generation prevents test data collisions
- Static shared state with guard flags ensures safe dependent step execution
- ExtentReports (dark theme) provides detailed pass/fail reporting with scenario-level logging
- All test data loaded from config.properties — zero hardcoded values in test code

7.2 Future Enhancements

1. Add JSON Schema validation using RestAssured's matchesJsonSchemaInClasspath() for structural response validation
2. Integrate with CI/CD pipeline (Jenkins / GitHub Actions) for automatic API regression on every commit
3. Add Allure Reports as an alternative to ExtentReports for richer test visualization
4. Enable parallel execution once state-sharing is refactored to use Cucumber dependency injection (PicoContainer) instead of static fields
5. Expand coverage to include authentication token-based APIs and header validation
6. Add response time assertions to validate API performance benchmarks

Appendix — Dependencies (pom.xml)

Dependency	Version	Purpose
io.rest-assured : rest-assured	5.x	HTTP request building and response parsing
io.cucumber : cucumber-java	7.x	BDD step definition annotations
io.cucumber : cucumber-testng	7.x	TestNG integration for Cucumber runner
org.testng : testng	7.x	Test runner and assertions
com.aventstack : extentreports	5.x	HTML test reporting with Spark dark theme
org.apache.logging.log4j : log4j-core	2.x	Structured console and file logging
com.precision : common (module ref)	1.0	ConfigReader, LogManager, ExtentReportManager

— End of Document —